

리눅스마스터 2급 1차 기본서

resources/ 폴더에 있는 족보·정리 자료(PDF, HWP, DOCX, HTML)를 종합·정리하여 작성한 개념 학습용 기본서입니다. 시험 범위 전반(리눅스 개요, 설치와 부팅, 파일시스템, 사용자 관리, 기본/텍스트 명령어, 셸, 네트워크, 시스템 관리, 라이선스)을 이론 중심으로 정리했습니다.

목차

1. 리눅스 개요
2. 라이선스와 배포판
3. 설치와 파티션
4. 부팅과 부트로더(LILO/GRUB)
5. 파일시스템과 디렉토리 구조
6. 사용자와 그룹 관리
7. 기본 파일·디렉토리 명령어
8. 텍스트 처리 명령어
9. 입출력 리다이렉션과 파이프
10. 셸과 환경변수
11. 네트워크 명령어
12. 시스템 관리(종료·재시작·런레벨·시간)
13. 자주 나오는 헛갈리는 포인트 정리

1. 리눅스 개요

1.1 리눅스의 탄생과 역사

- **리눅스(Linux)** = **리누스(Linus)** + **유닉스(Unix)**의 합성어. 유닉스를 모델로 개발됨.
- **1991년**, 핀란드 헬싱키 대학의 **리누스 토발즈(Linus Torvalds)**가 처음 개발하여 인터넷에 공개.
 - 초기 버전은 0.01로, 가장 기본적인 커널만 포함.
 - 리누스 토발즈는 유닉스의 **POSIX 호환**을 기반으로 하는 **커널(Kernel)**을 개발함.
- **유닉스(UNIX)**: 1970년대 초반 벨 연구소의 **켄 톰슨(Ken Thompson)**, 데니스 리치 등이 처음 개발한 범용 다중 사용자·시분할 운영체제. 이후 대부분 OS의 모태가 됨.
- **미닉스(MINIX)**: 교육용 유닉스로, 리눅스 개발의 시발점(모델)이 된 운영체제. 앤드류 타넨바움이 만듦.

1.2 GNU 프로젝트와 리처드 스톨만

- **리처드 스톨만(Richard Stallman)**: **GNU 프로젝트**의 리더이자, **자유소프트웨어재단(FSF, Free Software Foundation)**의 설립자 겸 회장. (<http://www.gnu.org>)
 - FSF: 컴퓨터 프로그램의 복제·배포·개작을 위한 소스 코드 이용에 대한 제한을 철폐하는 것이 목적.
 - 스톨만은 **HURD**라는 커널을 만들었으나 미완성 상태였고, 이후 토발즈의 리눅스 커널을 만나며 현재의 (GNU/)리눅스 운영체제가 완성되는 계기가 됨.
- **GNU**: "**G**NU's **N**ot **U**nix"(GNU는 유닉스가 아니다)의 재귀적 약자. 자유에 대한 구속을 반대하며 프로그램을 자유롭게 사용하도록 하자는 운동/프로젝트.
 - 많은 GNU 프로그램이 **GCC**로 컴파일됨. GNU 프로젝트는 UNIX를 개발한 프로젝트가 **아님**(UNIX는 벨 연구소가 개발).

- 리눅스라는 OS의 정확한 이름은 **GNU/Linux**이며, "리눅스"는 엄밀히는 **커널**만을 지칭.
- **리눅스 배포판**을 구성하는 3요소: **리눅스 커널 + 셸(Shell) + GNU 표준 유틸리티**. (허드(Hurd)는 배포판 구성 요소가 **아님** — 스톨만이 만든 별개의 커널)
- 리눅스에 포함된 대부분의 기본 유틸리티를 제공하는 곳: **GNU**.
- **GNU 프로젝트 산물 여부**: **gcc**, **emacs**, **gzip**, **GNOME**(데스크톱 환경)은 GNU 프로젝트 소속. 반면 **KDE**는 **GNU 프로젝트가 만든 것이 아님**(별도 프로젝트) — "GNU 프로젝트와 관련 없는 것"을 고르는 문제에서 KDE가 정답으로 나올 수 있음.

1.3 리눅스 초기 발전 타임라인 (자주 나오는 오답 포인트)

- **1991년 9월**: 커널이 처음 인터넷에 공개됨. 이 시점에는 **네트워킹 기능이 없었음**.
- **커널 0.01 등 초기 버전**: **단일 프로세서(Single Processor)** 환경에서만 동작 — 최초 버전부터 멀티프로세서(SMP)를 지원했다는 설명은 **틀림**.
- **1992년 10월**: 로스 비로(Ross Biro)가 불완전한 TCP/IP 코드를 구현하여 커널에 기여(흔히 **NET-1**로 불림) — 리눅스에 네트워킹 기능이 들어가기 시작한 계기.
- **1996년, 커널 2.0**: **멀티프로세서(SMP) 지원**이 본격적으로 도입됨.
- 참고로 **웨어웨어(Shareware)**는 일부 기능을 제한한 유료 구매 유도용 공개 소프트웨어로, 리눅스는 웨어웨어가 **아니며** POSIX 규격을 따르는 완전한 자유/오픈소스 운영체제.

1.4 리눅스의 기술적 특징

- **다중 사용자(Multi-user), 다중 처리(Multi-tasking/Multi-processing)** 시스템, 높은 **안정성·신뢰성**, 폭넓은 하드웨어 지원.
- **POSIX 표준**을 따르는 운영체제.
- 최상위 디렉터리 **/ (root)**를 기준으로 하위 디렉터리가 존재하는 **계층적(트리) 파일 구조**.
- **모든 장치를 파일로 관리**(장치 파일).
- **동적 라이브러리** 지원: 프로그램 특정 기능(루틴)을 실행 파일 내부에 포함하지 않고, 실행 시점에 가져다 사용 → 메모리를 효율적으로 사용, 효율성이 높음. (↔ 정적 라이브러리: 실행 파일에 라이브러리를 모두 포함)
- **리다이렉션**으로 입출력을 전환 가능, **파이프로 명령어 연결** 가능.
- 커널은 **C 언어**로 작성됨 (COBOL 아님 — 자주 나오는 오답 포인트).
- 애플·윈도우즈 NT 등에서 쓰는 다양한 파일시스템 지원, TCP/IP 네트워킹 지원, 멀티 프로세서 지원.
- 소스가 100% 공개 — 커널뿐 아니라 대부분의 응용프로그램 소스도 공개. 배포판 회사도 자유롭게 개발·수정·재배포 가능(공개·수정·배포 금지 아님).
- **커널**: 운영체제의 핵심 기능을 수행. 좁은 의미의 운영체제. 하드웨어(입출력 포함)를 관리하고 모든 응용프로그램의 실행 환경을 만들어 관리함.
 - 커널 버전 확인: **uname -r**
 - 커널 관련 사이트: <http://www.kernel.org>

2. 라이선스와 배포판

2.1 자유 소프트웨어와 오픈소스의 개념

- **자유 소프트웨어의 특징**
 - 상업적 목적으로 사용 가능
 - 소스 코드를 임의로 개작 가능
 - 소스 코드 수정 시 반드시 소스 코드를 공개해야 함

- ❌ "무료로 얻은 소스로 만든 프로그램은 무료로만 배포해야 한다" → **틀린 설명**(유료 판매도 가능. 다만 소스코드는 함께 공개해야 함)
- **오픈소스 소프트웨어**의 본질
 - GPL 등의 라이선스를 따름
 - 소스 코드까지 공개된 소프트웨어
 - Apache, GMP, PERL 등이 대표적 예
 - ❌ "프로그램의 소유권이 자유롭다" → **틀린 설명**(저작권/소유권은 존재하되, 소스 이용에 제한이 없는 것)

2.2 GPL 계열 라이선스

라이선스	발표 시기	핵심 특징
GPL v1	1989년 1월	최초의 GPL
GPL v2	1991년 6월	특허로 인해 추가 비용을 지불하거나 소스 공개가 불가능해 바이너리만 배포 하는 경우, 소스뿐 아니라 바이너리 배포 자체를 금지 . LGPL과 관계있음
GPL v3	2007년 6월 29일	-
LGPL (Lesser GPL)	-	라이브러리 형태로 사용 시, 그 라이브러리를 "이용"만 하는 프로그램(링크해서 쓰는 프로그램)은 소스 공개 의무가 없음. 단, LGPL 소스 코드 자체를 직접 수정했다면 그 부분은 공개해야 함. 독점적 소프트웨어와 결합 가능

GPL(GNU General Public License) 기본 정신 (FSF 제정):

- 누구나 어떤 목적으로든 기존 프로그램을 사용할 수 있다.
- 복사·수정 가능하며 재가공하여 판매도 가능하다.
- 배포(유·무료 불문) 시 **반드시 소스 코드를 함께 배포**해야 하며, 재가공된 프로그램도 동일하다.
- 파생 프로그램도 동일한 라이선스를 가져야 한다(강한 전염성/Copyleft).
- GNU 프로그램이 "공짜"라는 뜻은 아님(유료 판매 가능, 단 소스 공개는 필수).

2.3 그 외 주요 라이선스 비교

라이선스	소스 공개 의무	특징 / 대표 예
BSD	수정 후 공개 의무 없음 (공개해도 되고 안 해도 됨)	상용 프로그램에도 자유롭게 사용 가능. 재배포가 "의무"는 아님. 채택 예: Apple iOS/macOS의 다윈 (Darwin) 커널 은 FreeBSD·NetBSD의 BSD 코드를 다수 차용
MIT	공개 의무 없음	매사추세츠공대(MIT) 기원, "X11 License"라고도 함. 누구나 개작 가능, 수정본 재배포 시 소스 비공개 가능(가장 느슨함). 예: X Window System, jQuery, Node.js, Ruby on Rails, Visual Studio Code
Apache	공개 의무 없음(단, 재배포 시 라이선스 사본 포함 + 출처 명시 는 필요)	상업적 사용 가능. 예: Hadoop

라이선스	소스 공개 의무	특징 / 대표 예
MPL (Mozilla Public License)	MPL 코드 자체를 수정한 경우 그 부분은 공개(+저작자에게 수정 사실 고지)해야 하나, 결합된 다른 코드/실행파일은 독점 라이선스로 배포 가능(부분 공개 의무)	넷스케이프의 웹브라우저 소스 공개 목적으로 제정. 소스 코드 라이선스와 실행파일(바이너리) 라이선스를 분리 관리할 수 있음. 예: Firefox

- 소스 수정 시 **공개 의무가 없는** 조합: **BSD, Apache, MIT**
- **일부(제한적) 공개 의무**: LGPL, MPL
- 아파치 라이선스 소프트웨어 예: **Hadoop**

BSD 계열 OS 파생 관계(참고): FreeBSD(완전 무료) → TrueOS, GhostBSD, DesktopBSD, MidnightBSD, 애플 다윈으로 파생 / NetBSD(뛰어난 이식성 — 임베디드부터 구형 하드웨어까지 지원)에서 OpenBSD(보안 특화)가 파생.

2.4 리눅스 배포판(Distribution)

배포판 = 커널 + 셸 + GNU 유틸리티 + 수많은 오픈소스 프로그램을 모아, 설치·관리를 쉽게 하도록 패키징한 것.

계열별 분류

계열	대표 배포판
레드햇(Red Hat) 계열	RHEL(Red Hat Enterprise Linux), CentOS, Fedora, Oracle (Enterprise) Linux, 안녕 (AnNyung) Linux, 타이젠
데비안(Debian) 계열	우분투(Ubuntu), 민트(Linux Mint), Kali Linux, Knoppix, crunchBang Linux, Steam OS, Damn Small Linux, elementary OS, 기린, 하모니카
슬랙웨어 (Slackware) 계열	Slax(Slax OS), Vector Linux, DeLi Linux, Frugalware, Salix OS
맨드리바(Mandriva) 계열	Open Mandriva, Mageia, PCLinuxOS
안드로이드 계열	Android, Chrome OS, Remix OS, Polaris OS

⚠ **Knoppix는 데비안(Debian) 기반의 배포판입니다**(2000년 클라우스 크노퍼가 제작). CD/USB로 부팅해 설치 없이 쓰는 **라이브(Live) CD**의 대표 예. "슬랙웨어 계열에 속하는 것은?" 유형 문제에서 Knoppix는 **정답에서 제외되는(슬랙웨어 계열이 아닌) 보기**로 자주 등장하니, 슬랙웨어 계열 목록(Slax, Vector Linux, DeLi Linux, Frugalware, Salix OS 등)과 헷갈리지 않도록 주의하세요. ⚠ **SUSE는 RPM 패키지를 쓰지만 레드햇 계열이 아닌 독자 계열(때로 슬랙웨어 계열로 분류)의 상용 리눅스.**

우분투(Ubuntu): 영국 회사 **캐노니컬**이 **데비안** 리눅스를 기초로, 고유 데스크톱 환경(유니티 등)을 사용해 만든 배포판.

CentOS: **레드햇 엔터프라이즈 리눅스(RHEL)의 소스 코드를 그대로 가져와 빌드한 일종의 복제본(Clone)**. 2003년 12월 시작. 비용 부담 없이 자유롭게 배포 가능. 국내 리눅스 서버에서 많이 사용.

Fedora: 레드햇의 후원과 개발 공동체 지원 아래 배포. **6개월 간격**으로 새 버전 발표, 지원 기간이 다른 배포판보다 짧음.

RHEL(Red Hat Enterprise Linux): 대표적인 **상용판**(유료 라이선스) 리눅스. 레드햇이 기술지원 제공.

Scientific Linux: 페르미 국립 가속기 연구소가 만든, RHEL 기반의 자유/오픈소스 OS. 상용 배포판에 근접하게 설계. 주로 연구소/대학 사용.

보안/해킹 학습용 배포판

- **BackTrack:** 보안 테스트용 오픈소스 배포판. **2006년 2월 Knoppix 기반**으로 처음 만들어졌고, 해킹 관련 정보가 많아 "해킹머신"이라는 별명이 붙음. 이후 **2012년 8월** 우분투 기반으로 전환되어 마지막 버전이 출시됨.
- **Kali Linux:** BackTrack의 후속 배포판. **데비안 기반**. 다양한 해킹 도구를 내장하여 정보보안 학습에 유용.

국내 배포판: **SULinux**.

리눅스 커널 기반 OS: 안드로이드, 바다OS, Debian 등 (❌ **Minix**는 리눅스 커널 기반이 아닌 별개의 교육용 OS)

리눅스와 직접 관련 없는 정책: Copyright(저작권 자체는 라이선스 정책이 아님) — GPL, Free Software, LGPL과 구분해서 기억.

2.5 리눅스가 아닌 유닉스 계열 OS (참고·오답 함정용)

시험에는 "리눅스 배포판이 아닌 것 고르기" 유형으로 아래 OS들이 오답 보기로 자주 등장합니다.

- **macOS(과거 명칭 OS X):** Apple이 유닉스/다윈(Darwin) 기반으로 개발. **2001년 3월 24일** 최초 출시(OS X). **2016년 6월 13일 WWDC**에서 "macOS"로 명칭이 변경됨(macOS 시에라부터). "OS X"의 X는 **10**을 의미.
- **Solaris:** 썬 마이크로시스템즈가 **1990년** 개발한 유닉스 계열 OS. 이후 CDDL 라이선스 기반의 **오픈솔라리스**를 공개. SunOS 4 이후 "솔라리스 2"가 출시되면서, 이전 SunOS 4는 소급하여 "솔라리스 1"로 명명됨.
- 이 둘은 리눅스 커널을 사용하지 않는 **별개의 유닉스 계열 운영체제**이며, 리눅스 배포판이 아닙니다.

3. 설치와 파티션

3.1 설치 유형

배포판 설치 시 선택 가능한 유형: 워크스테이션, 서버, (사용자) 정의 설치, 업그레이드 등.

- ❌ **메인프레임**은 설치 유형에 **없음** (매우 자주 출제되는 오답 포인트)

3.2 설치 관련 주요 항목

- **사용자 보안 인증 관련 설정:** MD5, Shadow Password, NIS 등. (❌ **SSL**은 웹서버 보안인증으로 리눅스 "설치 시" 사용자 인증 설정 항목이 아님)
- **MD5:** 패스워드를 최대 **255자**까지 허용하여 강력한 보안 기능을 설정(단순 6~8자 제한을 넘어서는 긴 패스워드 지원).
- 설치 시 기본적으로 설정하지 않는 것(예): 스캐너
- 설치 디스크로 문제 발생 시 점검(복구) 모드: **Rescue installed system**
 - **Install or upgrade an existing system:** 설치/업그레이드 진행
 - **Install system with basic video driver:** 기본 비디오 드라이버로 설치
 - **Rescue installed system:** 이미 설치된 시스템 복구
 - **Boot from local drive:** 설치 없이 로컬 드라이브의 기존 시스템으로 부팅
- 설치 시 파티션 공간이 부족할 때 기존 파티션 크기를 줄여 공간을 확보하는 메뉴: **현재 시스템 축소하기**
- 파티션 분할이 안 된 여유 공간이 있을 때: **여유 공간 사용**

- 초보자에게 가장 쉬운 파티션 메뉴: **모든 공간 사용**
- 가상화(Xen/KVM) 사용을 위한 패키지 그룹: **Virtual Host**
- **yum** 사용을 위해 설정하는 항목: **리포지터리 설정**
- 가상화 프로그램(VirtualBox 등)에서 리눅스 설치 시 IPv4 기본 권장 설정: **자동(DHCP)**
- 네트워크 설정에서 부팅 후에도 카드가 연결되게 하려면 반드시 체크: **자동으로 연결**
- 한국 키보드 선택 메뉴: 보통 **U.S. English**로 선택(수험서 기준)
- 설치 시 나타나는 파일시스템 유형: **ext, ext2, ext3, ext4, vfat, swap, physical volume(LVM)** 등
- 특수 저장 장치(스토리지) 선택 관련 유형: **iSCSI, FCoE, SAN** 등 (❌ 일반 SCSI는 "기본 저장 장치"에 해당, 특수장치 선택자가 아님 — 지문 맥락에 유의)
- **파티션 분할 도구(GUI): Disk Druid** — 레드햇 계열 설치 과정에서 사용하는 수동 파티션 분할 도구. "파티션 ↔ Disk Druid" 조합이 올바른 짝(❌ "MBR ↔ Disk Druid", "MBR ↔ LILO"는 용어와 톨의 짝이 잘못됨)
- 리눅스를 정상 설치했을 때 **기본으로 생성되는 파일시스템: swap 파일시스템, 하나 이상의 Ext 계열(네이티브) 파일시스템, proc 파일시스템** (❌ **RAM 드라이브(romdrive) 파일시스템**은 기본 생성 대상이 아님)

3.2-1 설치 시 선택하는 패키지 그룹

패키지 그룹	설명
Web Server	Apache HTTP 등 웹 서버 구축용 도구 모음
Database Server	MySQL, PostgreSQL 등 데이터베이스 서버. (PostgreSQL은 PostgreSQL Global Development Group이 관리하는 완전 무료 오픈소스 DBMS로, MySQL이 오라클로 인수된 이후 대안으로 채택이 늘어나는 추세)
Virtual Host	가상화 시스템 구축용. Xen (하나의 컴퓨터에서 여러 OS를 설치·운영하는 가상화 기술), KVM (Kernel-based Virtual Machine, 커널 기반 가상머신) 사용 시 선택
Software Development Workstation	gcc 등 소스 컴파일 도구를 포함한 개발 환경

3.3 파티션(Partition)의 개념

- **파티션**: 하나의 물리적 디스크(하드디스크)를 여러 개의 **논리적** 디스크로 나누는 것.
- 파티션을 나누는 이유: 파일시스템 점검 시간 단축, 백업 용이, 특정 파티션 보호, 안정성(루트 파티션 손상 시에도 다른 파티션 자료는 보존).
- 파티션 분할의 장점: 파티션 단위로 다양한 정책 설정 가능, 여러 운영체제 사용 가능.
 - ❌ "파티션을 분할하면 부팅이 빨라진다" / "여러 운영체제의 부트로더를 사용할 수 있다" / "파일 크기가 커지면 다른 파티션을 활용할 수 있다" → 모두 **틀린 설명**(자주 함정으로 나옴)

파티션의 종류

- **주 파티션(Primary Partition)**: 하나의 디스크에 **최대 4개**까지 생성 가능. 부팅 가능한 파티션으로 디스크에 최소 1개는 있어야 함. 운영체제를 설치하는 곳을 보통 **루트(/) 파티션**이라 함.
- **확장 파티션(Extended Partition)**: 주 파티션 4개 중 **1개**를 확장 파티션으로 지정 가능(하나의 물리 디스크에 확장 파티션은 **1개만** 사용 가능). 확장 파티션 자체에는 데이터를 직접 저장할 수 없고, 그 안에 **논리 파티션**을 만들어야 사용 가능.
 - 5개 이상의 파티션이 필요하면 확장 파티션 선언이 필요.
 - 확장 파티션을 선언해야 논리 파티션을 만들 수 있음.
 - ❌ "확장 파티션은 2개 이상 설정 가능" → **틀림**(1개만)

- **논리 파티션(Logical Partition):** 확장 파티션 내부에 여러 개 생성 가능. **최대 12개**까지 생성 가능. 논리 파티션 번호는 **5번부터** 시작.
 - 주 파티션을 2개 사용 후 확장 파티션을 선언했더라도, 논리 파티션 번호는 여전히 **5번부터** 시작(4번부터 시작 **✗**).
 - 확장 파티션을 사용할 경우 실질적으로 사용 가능한 주 파티션 수는 **3개**(나머지 1개는 확장 파티션이 차지).
- **파티션 선언 순서:** 주 파티션 → 확장 파티션 → 논리 파티션
- **파티션 번호:** 주/확장 파티션 최대 번호 **4**, 논리 파티션 최소 번호 **5**
- 파티션은 **실린더(Cylinder)** 를 기준으로 나뉨. **fdisk**로 분할 시 실린더가 기준.
- 파티션 분할 상태 확인 파일: **/proc/partitions**
- 파티션 분할 프로그램: **fdisk, parted, kpartx, Disk Druid** 등 (**✗ partprobe**는 파티션 테이블을 커널에 재인식시키는 명령이지 분할 프로그램이 아님)

3.4 파티션·디스크 장치명

- **IDE 방식:** **/dev/hda, /dev/hdb, /dev/hdc ...** (디스크 개수만큼 a,b,c...), 파티션은 뒤에 숫자(**hda1, hda2...**)
 - Primary Master = hda, Primary Slave = hdb, Secondary Master = hdc, **Secondary Slave = hdd**
 - **/dev/hda:** 첫 번째 IDE HDD
- **SATA/SCSI 방식:** **/dev/sda, /dev/sdb ... + 숫자(sda1, sda2...)**
 - S-ATA 디스크 사용 중 USB 메모리를 추가 연결하면 다음 알파벳(**/dev/sdb** 등)으로 인식
- **GRUB 표기:** 디스크 종류(IDE/SATA/SCSI) 구분 없이 **hd0, hd1...** 로 표기, 파티션은 **hd0,0, hd0,1...** (**hd0,0** = 첫 번째 디스크의 첫 번째 파티션, **(hd0,3)** = 첫 번째 디스크의 **4번째** 파티션)
- IDE 케이블: 1개 케이블에 2개 장치 연결 가능 → 첫 번째 케이블=**Primary**, 두 번째 케이블=**Secondary** / 각 케이블에서 먼저 연결된 것=**Master**, 나중 것=**Slave**. E-IDE 디스크를 Secondary Slave로 연결 시 **/dev/hdd**로 인식(4번째 장치).

3.5 파티션/디렉토리 배치 권장

- 최소 파티션: **/(루트) + swap** 2개.
- 파티션 분할을 권장하는 영역: **/tmp, /usr, /var, /home, /boot** 등
 - **✗ /etc**는 분할 권장 영역이 **아님**
- 일반적으로 권장하는 조합 예: **/, SWAP, /usr, /var / /, SWAP, /home, /var / /, SWAP, /boot, /home**
 - **✗ /, SWAP, /etc, /var** → **틀림**(/etc 분할은 권장 대상 아님)
- **스왑(swap) 파티션**
 - 하드디스크 일부를 메모리처럼 사용하는 기술(가상 메모리 역할).
 - 스왑 파티션의 ID 번호는 **82** (일반 데이터 파티션은 보통 83).
 - 전통적으로 물리 메모리(RAM)의 **2배** 크기를 권장했음(과거 기준. 현재는 RAM 크기·용도에 따라 유동적으로 권장). 예) RAM 1GB → 권장 스왑 2GB.
 - 리눅스에서 스왑은 사실상 **반드시 분할**하는 것이 일반적(가상 메모리 역할 담당).
 - **✗ "스왑 파티션은 용량 제한이 없다"** → 시험에서는 **틀린 설명**으로 자주 출제(관례적 권장치가 있다는 취지).

3.6 파일시스템 유형 정리

- **저널링(Journaling) 파일시스템:** **ext3, ext4, JFS, XFS, ReiserFS** — 데이터 복구가 용이하고 복구 확률이 높음. (Ext2보다 읽기/쓰기 신뢰성·복구성이 우수)
 - **✗ ext2**는 저널링을 지원하지 않음(비저널링).

- **vfat**: 긴 파일명을 지원하는 윈도우 계열 파일시스템 타입(마이크로소프트 관련).
- **네트워크 파일시스템: SMB, NFS, CIFS** (❌ **JFS**는 로컬 저널링 파일시스템이며 네트워크 파일시스템이 아님 — 자주 헷갈리는 포인트)
- **NTFS**: 마이크로소프트 윈도우의 파일시스템으로, 리눅스가 기본적으로 자체 지원하는 파일시스템 목록(**ext, swap, proc** 등)에는 포함되지 않음. (참고: 오래된 시험 자료에는 "리눅스가 지원하지 않는 파일시스템 = NTFS"로 단순화되어 있으나, 실제로는 **ntfs-3g**라는 별도 드라이버나 최신 커널의 **NTFS3** 드라이버를 통해 읽기·쓰기가 가능합니다. 시험에서는 "기본 제공 여부"를 묻는 취지로 이해하면 됩니다.)
- **LVM(Logical Volume Manager)**: 여러 개의 디스크(파티션)를 하나로 묶어 사용하는 기술. 사용 중 파티션 크기를 줄이거나 늘릴 수 있음. 설치 시 파일시스템 유형에서 "physical volume(LVM)"으로 표시됨. 일부 디스크에 물리적 오류가 발생해도 데이터 손실 없이 정상 사용이 가능하도록 구성할 때 선택.
 - 구성 단계: **PV(Physical Volume, 물리 볼륨) → VG(Volume Group, 볼륨 그룹) → LV(Logical Volume, 논리 볼륨)**
 - 예: 10GB 디스크 + 20GB 디스크를 하나의 VG(30GB)로 묶은 뒤, 다시 15GB+15GB의 LV 두 개로 나눠 사용 가능. 볼륨 확장/축소를 **데이터 이전 없이** 유연하게 할 수 있는 것이 핵심 장점
- **RAID(Redundant Array of Independent/Inexpensive Disks)**: 여러 하드디스크에 데이터를 중복·분산 저장하여 안정성/성능을 높이는 기술.
 - **하드웨어 RAID**(전용 컨트롤러 하드웨어 사용) vs **소프트웨어 RAID**(운영체제 소프트웨어로 제어)
 - 소프트웨어 RAID(예: RAID1 미러링, RAID5)로 구성하면 디스크 일부에 물리적 오류가 발생해도 데이터 손실 없이 계속 운영 가능
 - 다수의 디스크·파티션을 하나로 묶는 기술의 조합 예: **LVM, RAID** (ext3/ext4는 "묶는 기술"이 아니라 파일시스템 자체)
- **Kdump**: 시스템 충돌(크래시) 시 정보를 수집해 원인을 규명하는 자료(덤프)를 제공하는 기능. 사용하려면 **설치 시 기능 사용 여부 선택 + 물리적 메모리 할당**이 필요.
- **NTP(Network Time Protocol)**: 네트워크를 통해 날짜/시간을 동기화할 때 사용하는 서버(프로토콜).

3.6-1 디스크 장치명 응용 예시

- S-ATA 디스크 1개(**sda**)가 장착된 상태에서 USB 메모리를 연결하면, USB도 SATA 방식으로 인식되어 **/dev/sdb**로 잡힘(뒤 알파벳은 인식 순서).
- **/dev/sda6** 해석: 주 파티션이 1~3번(3개)이고 4번이 확장 파티션이라면, **5번·6번은 논리 파티션** → **sda6**은 두 번째 논리 파티션을 의미(논리 파티션은 항상 5번부터 시작하므로 6번=2번째).

3.7 파일시스템 점검·복구, /etc/fstab

- **fsck**: 파일시스템을 점검·복구하는 명령. 오류가 발견된 파일은 **/lost+found**에 저장됨.
 - **-A**: **/etc/fstab**에 등록된 모든 파일시스템 검사
 - **-t [파일시스템종류]**: 점검할 파일시스템 타입 지정 (예: **fsck -t ext3**, 또는 **fsck.ext3**처럼 직접 호출)
 - **-f**: 정상으로 보여도 강제 검사
 - **-y**: 문제 발견 시 자동으로 "예(수정)" 처리
 - **-n**: 문제를 발견해도 수정하지 않고 내용만 출력
 - **-v**: 상세 정보(verbose) 출력
 - 예: **fsck -f -v /dev/sdb**
- **e2fsck**: (ext 계열을 포함한) 리눅스 파일시스템 점검·복구 명령. inode, 블록, 크기, 디렉터리 구조/연결, 파일 링크 정보, 전체 파일 개수, 사용 중 블록 등을 검사.
 - **-p**: 자동 복구, **-y**: 모든 질문에 예, **-n**: 모든 질문에 아니오, **-c**: 배드 블록 검사, **-f**: 정상 파일시스템도 강제 검사

- **종료 코드:** 0=에러 없음(정상), 1=파일시스템 오류를 복구함, 2=복구 후 재부팅 필요, 4=문제를 발견했으나 복구하지 않음, 8=실행(operational) 에러, 16=사용법/문법 에러, 32=사용자에 의해 작업 취소, 128=공유 라이브러리 에러
- **/etc/fstab**의 6개 필드(부팅 시 자동 마운트할 파일시스템 목록):
 1. 장치(볼륨 레이블 또는 UUID)
 2. 마운트 포인트(디렉터리)
 3. 파일시스템 종류 — 현재 커널이 지원하는 파일시스템 목록은 **/proc/filesystems** 에서 확인 가능. (참고: **CentOS 7부터 기본 파일시스템은 xfs**)
 4. 마운트 옵션 — **defaults**는 **rw,suid,dev,exec,auto,nouser,async**를 한 번에 지정하는 것과 같음
 5. 덤프(dump) 여부 — 0=덤프 불가, 1=덤프 가능
 6. 파일점검(fsck) 순서 — 0=검사 안 함, 1=루트 파일시스템(가장 먼저), 2=그 외 파일시스템
- **/proc/swaps**: 현재 설정된 스왑 파티션 및 사용 중인 스왑 파일 정보를 확인할 수 있는 가상 파일.

3.8 네트워크 인증 시스템

- **NIS, LDAP, Kerberos**: 네트워크상 사용자 인증에 사용되는 대표 시스템.
 - **NIS(Network Information Service)**: Sun Microsystems가 1980년대 중반 발표. 중요 시스템 DB 파일을 네트워크로 공유해 일관된 시스템 환경 제공.
 - **Kerberos**: 분산 컴퓨팅 환경에서 대칭키 암호로 사용자 인증을 제공하는 중앙집중형 인증 방식.
 - **LDAP(Lightweight Directory Access Protocol)**: TCP/IP 기반으로 디렉터리 서비스를 조회·수정하는 응용 프로토콜.
 - **✗ NFS**는 파일 공유 프로토콜로 사용자 "인증" 시스템이 **아님**.

4. 부팅과 부트로더(LILO/GRUB)

4.1 부트매니저(Bootloader) 개념

- 하드디스크 대용량화로 하나의 시스템에 2개 이상의 OS를 설치·구동하고자 하는 요구에서 등장.
- 부트로더는 여러 OS 중 어떤 것으로 부팅할지 선택하게 해주는 프로그램. 대체로 디스크에서 가장 먼저 읽히는 **MBR**에 설치되어 사용됨(단, **반드시** MBR에 설치해야 하는 것은 아님).
- **MBR(Master Boot Record)**: 첫 번째 디스크의 첫 번째 섹터에 **512바이트** 크기로 존재.
- 일반적인 멀티부트 환경 예: **MBR(LILO/GRUB)**, **/dev/hda1(Windows)**, **/dev/hda2(Linux)**, **/dev/hda3(swap)**

4.2 LILO(Linux LOader)

- 여러 OS를 선택할 수 있게 해주는 부트로더로, MBR에 위치. GRUB보다 **먼저** 개발됨.
- 반드시 MBR에 설치되어야 하는 것은 아님.
- **레드햇 계열에서만 제공되는 것이 아님**(여러 배포판에서 사용 가능했음) — "Redhat 계열에서만 제공된다"는 지문은 **틀림**.
- 컴퓨터 바이오스(BIOS) 정보를 참조하지 **않는**다는 설명은 **틀림**(LILO는 부팅 시 커널 위치를 찾기 위해 BIOS의 디스크 지오메트리 정보에 크게 의존하며, 이 때문에 지오메트리 불일치로 부팅이 실패하는 문제가 실제로 자주 발생했음).
- 설정파일: **/etc/lilo.conf**
 - **boot=/dev/hda**: LILO가 설치될 위치
 - **map=/boot/map**: LILO가 자동으로 생성하는 파일

- `install=/boot/boot.b` : 부트 섹터 위치 정보를 가진 파일
- `timeout=50` : 키보드 무입력 시 자동 부팅까지 대기(단위 1/10초 → 50이면 5초). (LILO의 `timeout` 단위는 GRUB과 다름에 주의)
- `label=` : 부팅 항목 이름(레이블) 지정 — "하드 디스크의 레이블을 지정"이 아니라 부팅 엔트리 이름
- LILO 버전만 확인하는 옵션: `lilo -v` (또는 `-V`)
- 일반적으로 설치 부팅 디스켓 제작 시 사용하는 부팅 이미지: **boot.img** (노트북용은 `pcmcia.img`)
 - `mount -t iso9660 /dev/cdrom /mnt/cdrom → cd /mnt/cdrom/images → dd if=boot.img of=/dev/fd0 bs=1440k`

4.3 GRUB(Grand Unified Bootloader)

- LILO 이후 등장한 부트로더. IDE 하드디스크를 **장착한 순서대로** 인식.
- GRUB에서도 부트 디스크를 통한 부팅을 **지원한다**(LILO/GRUB 비교 문제에서 "지원하지 않는다"는 지문이 오답으로 나오며 유의 — 자료마다 표현이 갈리므로 최신 해설 기준으로 **지원함**이 맞음).
- `timeout=10` : **10초** 대기 후 기본 설정된 운영체제로 자동 부팅.
- GRUB 부트 메뉴 진입 키
 - **[a]** : 커널 매개변수(parameter) **추가**(가장 손쉬운 방법). 싱글 모드 진입 등에도 사용.
 - **[e]** : 부트 메뉴 항목을 직접 **편집**(설정 내용이 지워졌을 때 등)
 - **[c]** : **상호대화식(명령줄) 모드**. 배시 셸과 유사한 환경에서 명령어 입력 가능.
- 커널이 들어있는 디렉터리: **/boot** (GRUB 관련 파일도 **/boot**에 위치)

4.4 부팅 과정과 런레벨(Runlevel)

- 시스템 부팅 순서: **ROM BIOS → HDD의 MBR → 리눅스 커널 이미지 로딩 → init 실행**
- **런레벨(Run Level)**: `init [숫자]` 로 전환 가능
 - **0** : 시스템 종료
 - **1** : **단일 사용자 모드**(싱글 모드) — 잊어버린 root 패스워드 복구 시 사용
 - **3** : **다중 사용자 텍스트 모드**(콘솔/텍스트 모드) — 서버 운영 시 X-Window 없이 메모리 절약 목적으로 흔히 사용
 - **4** : 일반적으로 사용되지 않으며, **사용자가 정의하여** 쓸 수 있는 레벨
 - **5** : **다중 사용자 그래픽 모드**(X-Window)
 - **6** : **재부팅**
 - CentOS7 계열: `multi-user.target` ≈ 런레벨 3, `graphical.target` ≈ 런레벨 5
- 재부팅 시 호출되는 런레벨(기본 런레벨) 정보를 담은 파일: **/etc/inittab** — 예: `id:5:initdefault:` 라고 되어 있으면 부팅 시 기본 런레벨이 **5(그래픽 모드)**. 3이면 텍스트(콘솔) 모드.
- 로그인 관련 안내 문구가 담긴 파일들
 - **/etc/issue** : 로그인 프롬프트 **전에** 보여지는 배너로, 배포판/커널 버전을 가장 손쉽게 확인할 수 있는 파일(예: `cat /etc/issue → CentOS release 6.9 (Final) Kernel \r on an \m`). 보안을 위해 이 내용을 변경/삭제해두는 경우도 있음
 - **/etc/issue.net** : **원격(네트워크) 접속** 시 보여지는 배너
 - **/etc/motd** : 로그인에 **성공한 뒤** 보여주는 공지/환영 메시지(예: 서버 점검 예정 공지)
- `reboot`, `shutdown -r`, `init 6` 은 모두 **재부팅**을 의미. `shutdown -h`, `halt`, `init 0`, `poweroff` 는 모두 **종료**를 의미(`reboot-init6`과 결과가 다름 — 4개 중 다른 하나 찾기 문제 단골 포인트).
- root/GRUB 패스워드 분실 시 리눅스 설치 디스크로 복구: **Rescue installed system** 모드 선택.

5. 파일시스템과 디렉토리 구조

5.1 기본 개념

- 리눅스에서는 디렉터리도 하나의 **파일**로 인식.
- 디렉터리는 **트리(Tree) 구조**로, 최상위 **루트(/)** 를 중심으로 하위 디렉터리(**/usr**, **/home**, **/etc** 등)가 존재.
- 각 파티션마다 별도의 루트 파일시스템이 존재하는 것이 **아니라**, 전체가 하나의 트리 구조로 통합됨(윈도우의 드라이브 문자 개념과 다름).
- 리눅스 파일은 **inode**(파일의 메타정보)와 **데이터 블록**을 가짐. 디렉터리 파일의 데이터 블록에는 그 안에 있는 파일들의 inode 값이 들어있음.

5.2 주요 디렉토리 정리

디렉토리	설명
/bin	기본 실행파일(외부 명령어) 위치. 사용자 명령어들이 실행 파일 형태로 저장. 어느 경로에서든 실행 가능하므로 굳이 이 경로로 이동해서 실행할 필요는 없음
/sbin	System Binary. 파일시스템 처리, 네트워크 인터페이스 설정, 시스템 초기화, 커널 모듈 관리 등 관리자용 명령 . root 계정만 사용 허가
/etc	Etcetera. 시스템 설정 파일 보관: passwd , group , printcap (프린터 목록), fstab (자동 마운트 파일시스템 등록), 네트워크 설정 등
/dev	Device. 마운트 지점 제공. 마우스·키보드·모니터·CD-ROM·하드디스크 등 주변장치를 파일로 처리 하여 저장
/lib	시스템/응용프로그램이 사용하는 대부분의 라이브러리. 임의 조작 금지 권장
/home	사용자 계정 생성 시 계정명과 동일한 디렉터리가 자동 생성되는 곳. 사용자의 "개인 공간". 다른 디렉터리 접근은 관리자 허가 필요
/root	root 사용자 전용 홈 디렉터리. 일반 사용자 출입 금지
/proc	커널과 프로세스를 위한 가상 파일시스템 (메모리상의 정보를 파일 형태로 제공). cpuinfo (CPU 정보), interrupts (인터럽트 목록), ioports (I/O 주소 목록), pci (PCI BIOS 정보), stat (시스템 통계), partitions (파티션 정보) 등
/boot	부팅에 필요한 파일, GRUB (부트로더), lost+found [부팅 관련] 등
/tmp	임시 파일 생성·삭제 공간(기본적으로 Sticky-bit 설정)
/var	Variable Data. 가동 중 생기는 각종 임시 파일 저장. /var/log 에 대부분의 로그 파일 , 전자메일, 프린트 관련(스풀링) 파일 저장
/usr	Universal System Resources. 시스템·응용프로그램에 필요한 파일 저장. 설치 시 가장 큰 용량 을 배정해야 하는 디렉터리. /usr/local 은 새 프로그램 설치 위치(윈도우의 Program Files와 유사)
/opt	Add-On(응용) 소프트웨어 패키지 설치 공간. 패키지 매니저가 설치/삭제 수행. 레드햇 계열은 기본 구성 안 함 . (❌ "/opt에는 각 장치에 필요한 socket 및 log 파일이 있다" → 틀린 설명, 이는 /var 관련 내용)
/lost+found	파일시스템 복구 (fsck)를 위한 링크 디렉터리. fsck 등에서 발견된 결함 파일 정보 보관
/mnt	마운트될 파일시스템의 마운트 포인트

⚠ `/include, /op` 는 리눅스 표준 디렉터리가 **아님**(문제에서 함정으로 자주 등장).

5.3 마운트(Mount)

- `mount` : 장치(파일시스템)를 특정 디렉터리에 연결.
 - `mount -t ext2 /dev/hdc1 /usr/local : /dev/hdc1을 /usr/local에 마운트`
 - `mount -a : /etc/fstab(또는 /etc/mtab)에 나열된 모든 파일시스템을 마운트`
- `umount` : 마운트 해제. 이미 사용 중(어떤 프로세스가 해당 디렉터를 사용 중)이면 언마운트가 되지 않음.
- `/etc/fstab` : 부팅 시 자동으로 마운트되도록 등록해두는 파일(파일시스템 테이블).
- `eject` : CD-ROM 등 장치를 열고 닫을 때 사용.

6. 사용자와 그룹 관리

6.1 계정 관련 파일

파일	필드 수	설명
<code>/etc/passwd</code>	7개	계정명:패스워드(x):UID:GID:설명:홈디렉토리:로그인셸
<code>/etc/shadow</code>	9개	실제 암호화된 패스워드 와 만료/유효기간 정보 저장. 일반 사용자는 읽기 권한도 없음 (root만 접근)
<code>/etc/group</code>	4개	그룹명:패스워드:GID:그룹에 속한 계정 목록
<code>/etc/gshadow</code>	3~4개	그룹 관리자 정보 등(!가 있으면 그룹 암호 없음을 의미)

- `/etc/passwd`의 2번째 필드가 `x`로 표시되는 것은 **패스워드가 `/etc/shadow`에 암호화되어 별도 저장되었다**는 뜻(패스워드가 `/etc/passwd`에 저장된 것이 **아님** — 자주 나오는 오답 포인트).
- 로그인 셸에 따른 의미
 - `/bin/bash` : 정상적으로 로그인 가능한 활성 계정
 - `/bin/false` : 로그인 불가능. ssh-셸,터널링,홈 디렉터리 사용 불가. FTP 등 프로그램도 사용 불가
 - `/sbin/nologin` : 로그인 불가(메시지 반환). ssh 불가하나 **FTP는 사용 가능**
- 계정·그룹 정보 관련 파일 정리
 - `/etc/login.defs`: 전체 사용자 계정의 **기본 정책**(메일 디렉터리, 패스워드 관련 설정, UID 최소/최대 값, 홈 디렉터리 생성 여부 등)
 - `/etc/default/useradd`: 계정 생성 시 적용되는 기본 정책(기본 홈 디렉터리 경로 등). `useradd -D`로 확인 가능
 - `/etc/skel`: 계정 생성 시 홈 디렉터리에 기본으로 복사해 줄 파일·환경설정 보관(bash 설정파일 등)
 - `/etc/motd`: 로그인한 사용자에게 공지사항(예: 서버 점검 예정)을 보여주는 파일
 - `/etc/profile`: 전체 사용자 로그인 시 적용되는 환경설정, 검색경로(PATH) 등을 지정. 일정 시간 미사용 시 강제 로그아웃(TMOU) 설정 등에도 관련
 - 개인 사용자 검색경로 지정: 홈 디렉터리의 `.bash_profile`

6.2 사용자 계정 관리 명령어

- `useradd` : 계정 생성(=root가 아닌 사용자에게 시스템 사용권을 부여하는 명령). 계정 추가 시 기본 설정 항목: 홈 디렉터리, 기본 셸, 그룹/GID 등 (**vi 에디터는 기본 설정 항목이 아님**)
 - `useradd -D` : 기본 정책 확인

- `useradd -f [일수]` : 비밀번호가 만료된 후 계정이 잠기기(비활성화)까지의 유예 일수 지정 (-e는 계정 자체의 만료 "날짜"를 지정하는 옵션으로 서로 다름, 주의)
- `useradd -d [홈경로] -g [기본그룹] -s [셸]` 계정명처럼 여러 옵션을 조합해 계정 생성 시점에 한 번에 지정 가능. 예: `useradd -d /home/userid -g users -s /bin/sh userid`
- ❌ **USERCREATE** 같은 명령은 존재하지 않음
- **passwd [계정명]** : 비밀번호 설정/변경. `root`가 다른 사용자(`choi`)의 패스워드를 바꿀 때: `passwd choi`
 - New password 입력 시 화면에 **암호가 표시되지 않음**(그대로 표시된다는 지문은 틀림 — 보안상 당연)
 - `passwd -l [계정]` : 계정 패스워드에 **잠금** 설정 → 일시적으로 로그인 차단. 내부적으로는 `/etc/shadow`의 패스워드 필드 맨 앞에 **!!**(또는 **!**)를 붙여 인증이 실패하도록 만드는 방식
 - `passwd -u [계정]` : 잠금 **해제**
 - `passwd -d [계정]` : 패스워드를 지워 **입력 없이 로그인**되게 함(주로 임시 계정 등에 사용, 보안상 위험할 수 있음)
 - `passwd -S [계정]` (대문자) : 계정의 패스워드 상태(설정일, 최소/최대 사용기간 등) 요약 정보 출력. 예: `passwd -S kaituser → kaituser PS 2014-05-13 0 99999 7 -1`
 - 계정 잠금-해제의 `usermod` 버전: `usermod -L`(잠금) / `usermod -U`(해제)
 - 최초 설정 후에도 언제든지 **변경 가능**(변경 불가라는 지문은 틀림)
- **usermod [옵션] [계정명]** : 계정 정보 수정
 - `-c` : 계정 설명(코멘트) 지정
 - `-d` : 홈 디렉터리 변경 (`-m`과 함께 쓰면 파일도 같이 이동)
 - `-e` : 계정 **만료일** 지정 → `/etc/shadow`에 반영. 예: `usermod -e 2020-12-31 ihduser`
 - `-g` : 기본 그룹(GID) 지정. 예: `usermod -g IHD ihduser`
 - `-G` : 보조 그룹 지정
 - `-l` : 계정명 변경(소문자 l). 예: `usermod -l kaitmember ihduser`
 - `-s` : 로그인 셸 변경. 예: `usermod -s /bin/bash ihd1`
 - `-u` : UID 지정
 - `-f` : 만료 후 잠금까지의 유예 기간 지정
- **userdel [계정명]** : 계정 삭제(옵션 없으면 계정 정보만 삭제, 홈 디렉터리는 유지)
 - `-r` : 홈 디렉터리-메일함 등 관련 파일까지 **재귀적으로 모두 삭제**(하위 디렉터리 포함). 시험에서 "홈 디렉터리의 하위 디렉터리까지 삭제하는 옵션"의 정답은 `-r`
 - `-f` : 계정이 로그인 중이거나 그룹 소유 문제가 있어도 **강제 삭제**
 - ⚠ 실제 GNU shadow-utils의 대문자 `-R`은 "하위 디렉터리 삭제"가 아니라 `--root CHROOT_DIR`(다른 루트 경로의 설정파일을 대상으로 명령 적용) 옵션입니다. 일부 오래된 족보 자료에 `-R`을 "홈 디렉터리 하위까지 삭제"로 잘못 설명한 경우가 있으니 주의하세요 — 시험 문제의 선택지에 `-r`(소문자)과 `-R`(대문자)이 함께 나오면 정답은 **소문자 -r**입니다
- **chage** : 사용자 패스워드 정보 출력 및 `/etc/shadow`의 날짜 관련 필드를 종합적으로 설정할 수 있는 명령
- **su [계정명]** : 다른 사용자로 전환. `su` - 또는 `su -l` 을 사용하면 실제 로그인한 것처럼 **환경까지 적용됨**(`-c`는 특정 명령 실행 옵션으로, "환경 적용" 옵션이 아님)
- **id** : 로그인한 사용자의 실제/유효 UID, GID 확인
- **groups [계정명]** : 특정 계정이 속한 그룹 확인. 예: `groups shadow879`
- **lslogins** : 시스템에 등록된 사용자 계정 정보를 **UID 순으로** 출력해주는 명령어
- **users / who / w** : 현재 시스템에 접속(로그인)한 계정 확인
 - `users` : 로그인된 사용자의 계정명만 나열(같은 계정이 여러 세션이면 중복 표시될 수 있음)
 - `who` : 접속 사용자 + 터미널·로그인 시간 (1줄 = 1세션 → `who | wc -l` 로 현재 접속자 수를 셀 수 있음)
 - `w` : 접속 사용자 + 현재 수행 중인 작업까지 표시(가장 정보가 많음)
- `/etc/shadow` 파일에는 **로그인 셸, UID, GID 정보가 없음**(이 정보들은 `/etc/passwd`의 필드에 있음) — `"/etc/shadow에 포함된 내용이 아닌 것"`을 고르는 문제에서 UID/GID/로그인 셸이 정답으로 나올 수 있음.

6.3 그룹 관리 명령어

- `groupadd [그룹명]` : 그룹 생성. `-g [GID]` 옵션으로 GID 지정 가능. 예: `groupadd -g 1001 ihd`
- `groupmod -n [새이름] [기존이름]` : 그룹 이름 변경. 예: `groupmod -n ihd kait`
- `groupmod -g [GID] [그룹명]` : GID 변경. 예: `groupmod -g 700 lms`
- `groupdel [그룹명]` : 그룹 삭제

7. 기본 파일·디렉토리 명령어

7.1 위치·이동 관련

- `pwd` : **P**rint **W**orking **D**irectory. 현재 작업 디렉터리의 **절대 경로** 출력
- `cd [경로]` : 디렉터리 이동
 - `cd` 또는 `cd ~` : 홈 디렉터리로 이동
 - `cd ..` : 한 단계 위(부모) 디렉터리로 이동
 - `cd -` : **직전에 있던 디렉터리**로 이동
 - `cd ../..` 는 `cd ..` 와 동일한 의미(둘 다 부모 디렉터리 이동) — 반면 `cd home`(상대경로, 하위 디렉터리 이동 시도)은 의미가 다름
- `tty` : 현재 접속한 터미널의 장치 파일명 확인 (예: `/dev/pts/0`)
 - `pts/0`(Pseudo Terminal Slave) = 가상 콘솔. 가상 콘솔은 보통 6개(`tty1~6`), 7번째는 X-Window
 - 다른 가상 터미널로 전환: `Ctrl+Alt+F1~F6`(텍스트 콘솔), `Ctrl+Alt+F7`(X-Window). 현재 터미널 외 추가 터미널 사용: `[ALT]+[F2]`
- `stty` : 현재 터미널의 설정 정보 확인/변경(speed, baud 등)
- `whereis [명령어]` : 명령어의 **실행파일·소스·매뉴얼(man)** 위치를 모두 보여줌
- `which [명령어]` : 명령어의 **실행 파일 경로**(위치)만 찾아줌 — `which` 는 **PATH** 환경변수와 관련이 깊음
- `locate [패턴]` : 사전에 생성된 DB(색인)를 기반으로 파일을 빠르게 검색. **cron 데몬**이 주기적으로 갱신하는 DB를 사용.
 - 최근 추가한 파일/디렉터리가 안 보이면: `updatedb` 로 DB를 갱신
- `manpath` : man 페이지가 위치한 경로들을 출력
- **man 섹션 번호(1~9)**: 같은 이름이라도 섹션에 따라 다른 문서를 보여줌

섹션	내용
1	일반 명령어(man의 기본값)
2	시스템 호출(프로그래밍)
3	라이브러리/함수 호출(프로그래밍)
4	디바이스(<code>/dev</code>) 관련
5	파일 형식 (설정파일 포맷)

섹션	내용
6	게임
7	기타(매크로 패키지, 규약 등)
8	시스템 관리 명령(root 권한 필요)
9	커널 관리(비표준)

- `/etc/passwd` 파일의 **7개 필드**에 대한 설명을 보려면 파일 형식 섹션인 `man 5 passwd` (또는 `man /etc/passwd`) 사용

7.2 목록·정보 확인

- `ls` [옵션] : 디렉터리 내용(목록) 확인 (DOS의 `dir`과 유사)
 - `-a`, `--all` : 숨김파일 포함 전체 출력
 - `-l` : 상세 정보(퍼미션, 소유자, 크기, 날짜 등) 출력
 - `-d` : **디렉터리 자체**의 정보만 출력 (예: `ls -ld /디렉터리명` → 특정 디렉터리의 퍼미션 확인)
 - `-s` : 파일 크기(KB) 기준 출력
 - `-t` : 최근 수정 파일부터 정렬
 - `-r` : **알파벳 역순** 정렬(기본은 오름차순)
 - `-R` : 하위 디렉터리까지 재귀적으로 출력
 - `-F` : 파일 종류에 따라 특수문자(/, *, @ 등) 덧붙여 출력
 - `-al` 처럼 여러 옵션 동시 사용 가능
 - 예: `mkdir .fileA` 로 만든 숨김 디렉터리는 `ls -a` 로 확인 가능, `cd .fileA` 로 이동도 가능
- `file` [파일명] : 파일의 **종류/형식**을 확인. 예: `file /etc/resolv.conf`, `file -i` 옵션으로 MIME 타입 확인
- `wc` [옵션] [파일명] : 파일의 줄(행)·단어·문자 수 출력
 - `-l` : 줄 수만, `-w` : 단어 수만, `-c` : 바이트(문자) 수만, `-m` : 문자 수만, `-L` : 가장 긴 줄의 길이
 - 옵션 없이 실행 시 출력 순서: **줄 수 단어 수 문자 수**

7.3 생성·복사·이동·삭제

- `mkdir` [옵션] [디렉터리명] : 디렉터리 생성(생성한 사용자 소유)
 - `-p` : 존재하지 않는 상위(부모) 디렉터리까지 한 번에 생성. 예: `mkdir -p /usr/bin`
- `touch` [파일명] : 파일의 **접근/수정 시각(타임스탬프)** 갱신. 파일이 없으면 **빈 새 파일**을 생성. 이미 존재하는 파일에 실행하면 **내용은 그대로 유지**되고 시간 정보만 바뀜(내용이 사라지지 않음)
 - `touch MMDDHHmm 파일명` 형태로 특정 시간 지정 가능
 -  "touch로 파일의 Change Time(변경 시간)을 과거로 바꿀 수 있다" → **틀림**. Change Time은 시스템이 자동 기록하는, 사용자가 임의 조작할 수 없는 절대적 시간 개념(파일에 부여되는 타임스탬프 중 "절대 불변"의 시간)
- `cp` [옵션] 원본 대상 : 파일/디렉터리 복사
 - `-f` : **확인 없이 강제로 복사**(동일 파일명이 있으면 기존 파일을 지우고 덮어씀)
 - `-i` : 덮어쓰기 전 확인(질의) — `-f`와 반대로 "물어보는" 옵션이므로 혼동 주의
 - `-r/-R` : 디렉터리(하위 포함) 복사 시 필요
 - `-p` : 원본의 소유권·권한·타임스탬프 등 속성을 유지하며 복사
 - `-a` : 속성과 링크 정보까지 그대로 유지하며 복사(아카이브 모드), `-d` : 심볼릭 링크 자체를 링크로 유지(원본 파일 대신 복사되는 것 방지)
- `mv` 원본 대상 : 파일 이동/이름 변경. 디렉터리 대상 작업 시 **특별한 옵션이 필요 없는 명령**

- `mv -i`: 덮어쓰기 전 사용자에게 확인 질의
- `rm` [옵션] 파일/디렉터리 : 삭제
 - `-i`: 삭제 전 파일마다 하나씩 확인(질의)
 - `-I` (대문자): 파일을 4개 이상 삭제하거나 재귀 삭제할 때만 **한 번만** 확인
 - `-r`: 디렉터리(하위 내용 포함) 삭제
 - `-f`: 확인 없이 강제 삭제(경고 메시지도 표시 안 함)
 - `-v`: 삭제되는 내용을 화면에 표시(verbose)
 - `-rf/-r -f`: 확인 없이 강제로 디렉터리 전체 삭제 → `rm -r [디렉터리]` 실행 시 **사용자 확인 없이 즉시** 디렉터리 내 파일 및 하위 디렉터리까지 **모두** 삭제됨
- `rmdir` [디렉터리명] : **비어 있는** 디렉터리만 삭제 가능
- `ln`: 링크 생성
 - 하드 링크: `ln 원본 링크명` — 동일한 데이터(inode)를 다른 이름으로 접근. **디렉터리에는 사용 불가**(일반 파일 대상)
 - 심볼릭 링크: `ln -s 원본 링크명` — 원본 경로를 가리키는 참조 파일. 일반 파일/디렉터리 모두 링크 가능

8. 텍스트 처리 명령어

8.1 내용 확인/출력

- `cat` [파일] : 파일 내용을 한 번에 모두 출력(병합에도 사용: `cat a.txt b.txt`)
 - `-n`: 줄 번호 추가
 - `-A`: 개행문자·탭 문자 등 **비출력 문자를 표시**(파일 안 숨은 문자 확인용)
- `more` [파일] : 화면 단위로 출력(페이지 이동은 **아래로만** 가능, 지나간 내용은 다시 못 봄)
- `less` [파일] : `more`의 확장판. **커서로 상하좌우 자유 이동** 가능(위/아래 모두 스크롤 가능), 대용량 파일에서 빠름
- `man` : 매뉴얼(도움말) 페이지 확인. `man`, `cat`, `more`, `less` 모두 "문서(파일) 내용을 읽는" 명령어군. `man/less`는 페이지 단위+이전 페이지 복귀 가능
- `head` [-n 숫자] 파일 : 파일의 **앞부분** 출력(기본 10줄)
- `tail` [-n 숫자] 파일 : 파일의 **뒷부분** 출력(기본 10줄)
 - `tail -f`: 로그 파일처럼 **동적으로 바뀌는** 내용을 실시간으로 계속 보여줌(follow)
- `clear` : 화면(터미널)을 지워 첫 줄에 프롬프트가 나오게 함(바이너리 파일을 `cat`으로 봤을 때 깨진 화면 정리용으로도 사용). (윈도우의 `cls`에 대응. 리눅스에는 `cls`가 없음)

8.2 비교

- `diff` 파일1 파일2 : 두 파일의 **차이점**(라인 단위)을 비교해 보여줌
 - `-i`: **대소문자를 구별하지 않고**(무시하고) 비교 — **!** "`-i`는 대소문자를 구별한다"는 설명은 **틀린 지문**입니다(정반대). `-i`는 대문자·소문자를 같은 것으로 취급합니다
 - `-b`: 공백 문자 차이 무시(하나 이상의 공백을 동일하게 취급)
 - `-w`: 공백을 무시하고 비교
 - `-e`: ed 에디터용 스크립트 생성
- `cmp` 파일1 파일2 : 두 파일을 **바이트 단위로** 비교해, 처음으로 다른 위치(바이트/줄)만 출력. 예 결과:
`com1.txt com2.txt differ: char 3, line 1`
- `comm` 파일1 파일2 : **행 단위로** 비교(정렬된 파일 기준으로 공통/차이 열 표시)
- (파일 비교와 무관한 명령: `gcc` — 컴파일러이지 비교 도구가 아님)

8.3 검색

- **grep** [옵션] 패턴 파일 : 텍스트에서 **특정 패턴(문자열)** 을 가진 줄을 찾아 출력
 - **-r** : 하위 디렉터리까지 재귀 검색. 예: `grep -r linuxmaster *`
 - **-v** : 패턴을 **포함하지 않는** 줄만 출력. 예: `cat /etc/passwd | grep -v linuxmaster → linuxmaster 문자열이 없는 행만 출력`
 - **-c** : 매칭된 줄의 **개수**만 출력
 - **-E** : 확장 정규식(OR 등). 예: `grep -E 'ihduser|yuloje' jalin.txt`
 - **-v ^#** : #으로 시작하지 않는 줄 출력. 예: `grep -v ^# vsftpd.conf`
 - **[abc]** 형태의 대괄호 패턴: a, b, c 중 **한 글자라도 포함**되면 매칭(문자 집합). 예: `grep -v [abc] job.txt → a-b-c 중 하나라도 들어있는 줄을 제외하고 출력`
- **find** [경로] [옵션] [찾는조건] [action] : 조건에 맞는 파일/디렉터리를 **직접 탐색**(느리지만 최신 상태 반영)
 - **-name** : 이름으로 검색. 예: `find / -name a, find / -name '*.txt' 2>/dev/null`(오류 메시지 숨김)
 - **-user** : 소유자로 검색
 - **-newer** : 특정 파일보다 최근/이전에 수정된 파일 검색
 - **-perm** : 권한(permission)으로 검색
 - **-size** : 크기로 검색
 - **-type** : 파일 종류(**d**=디렉터리, **f**=파일)로 검색
 - **-mtime +5** : 5일 이상 전에 수정된 파일 검색 (`find . -mtime +5 -print`)
 - **-exec 명령 {} \;** : 검색된 파일에 대해 명령 실행 (예: 찾은 후 즉시 삭제 `find -name '*.txt' -exec rm {} \;`)
 - **-ok 명령 {} \;** : 사용자 확인 후 실행
 - **-print** : 검색된 파일의 절대경로 출력(기본 동작)
 - **-ls** : 검색 결과를 상세(long) 형식으로 출력
- **locate** 패턴 : **사전에 만들어진 색인 DB**를 이용해 빠르게 검색(updatedb로 갱신 필요). find보다 빠름.

8.4 자르기/추출/변환

- **cut** [옵션] 파일 : 파일에서 원하는 부분(필드/문자 위치)만 잘라 출력
 - **-c [위치]** : 문자 위치로 자르기(콤마·하이픈 혼용 가능). 예: `cut -c 1,3` (1번째·3번째 문자만), `cut -c 1-3` (1~3번째 범위)
 - **-b [위치]** : **바이트** 위치 기준으로 자르기(멀티바이트 문자에서 **-c**와 결과가 달라질 수 있음)
 - **-f [필드번호]** : 필드로 자르기(콤마로 복수 지정, 하이픈으로 범위 지정 가능)
 - **-d [구분자]** : 필드 구분자 지정(기본은 탭). **-d** 는 보통 **-f**와 함께 사용
 - 예: `cut -d: -f 1,3 /etc/passwd → : 기준 1번째·3번째 필드 출력`
- **sort** [옵션] 파일 : 줄 단위 정렬(기본 오름차순, 문자 기준)
 - **-r** : 내림차순, **-n** : **숫자 값**으로 정렬(문자열 우선순위가 아닌 실제 수치 기준), **-t** : 구분자 지정, **-k** : 정렬 기준 열(필드) 지정, **-b** : 앞 공백 무시, **-f** : 대소문자 구분 안 함, **-u** : 중복 행 제거, **-m** : 파일 병합 정렬, **-o** : 결과를 파일로 저장
 - 예: `sort -t: -k3,3n -k1,2r /etc/passwd → 3번째 필드는 숫자로, 1~2번째 필드는 역순 정렬` (GNU 최신 방식 권장 옵션. 과거 `+2n -3` 식 표기는 구식 문법)
- **uniq** [옵션] 파일 : **연속으로 중복된 줄**을 하나로 합침(중복 제거). **-c** : 중복 횟수 함께 표시. (❌ "파일 내용을 줄 단위로 정렬하여 디스플레이"는 sort의 역할이지 uniq의 정의가 아님)
- **split** [옵션] 파일 : 큰 파일을 여러 개의 작은 파일로 분할

- 옵션 없이 실행 시 기본 **1,000줄** 단위로 분할, 분할된 파일명 뒤에 영문 2자리 추가(파일명aa, 파일명 ab...)
- **-b**: 바이트 단위 분할, **-l**: 줄 수 기준 분할, **-n**: 균등 분할, **-d**: 숫자로 접미사(0부터), **-a**: 영문 접미사 자릿수 지정
- 예: `split -b 10000 /etc/services`
- **strings [파일]**: 텍스트가 아닌(바이너리) 파일에서 사람이 읽을 수 있는 문자열만 추출해 출력(실행파일 등에서 활용)

8.5 기타 유틸리티

- **date [옵션]**: 현재 날짜·시간 출력/설정.
 - **date + "형식"**: +로 시작하면 **지정한 형식으로 출력**(설정이 아님). 예: `date +%Y %m %d`
 - %Y(연도 4자리) ↔ %y(연도 뒤 2자리) / %M(분) ↔ %m(월) / %D(%m/%d/%y 형식) ↔ %d(일) — **대소 문자에 따라 의미가 다르므로 주의**
 - +가 없이 **date [날짜시간문자열]** 형태로 실행하면 시스템 시계를 그 값으로 **설정**하려는 시도가 됨
 - **-d "문자열"**: 상대적인 날짜를 계산해서 보여줌(시스템 시간을 바꾸지 않음). 예: `date -d "-1 days"` (어제), `date -d "-1 weeks"`(1주 전), `date -d "+10 days"`(10일 후)
 - **-s "문자열"**: 시스템 시간을 직접 설정. 예: `date -s "2023-11-01 13:42:00"`
 - **✗ setterm**은 터미널 화면 속성(색상 등)을 설정하는 명령으로, 날짜/시간과 무관(오답 함정으로 등장)
- **cal [일] [월] [연도]**: 달력 출력. 옵션 없이 실행하면 **오늘 날짜가 속한 달**의 달력만 출력(1년 전체 아님).
`cal 6 1982` → 1982년 6월 달력.
 - **-y**: 지정 연도(또는 올해) **전체** 달력 출력
 - **-j**: 율리우스력(1월 1일부터 누적된 날짜 수)으로 표시
 - **-V**: cal 명령의 버전 정보 출력
- **time** 명령어: 해당 명령어의 **실행 소요 시간**을 측정(real/user/sys로 구분해 출력). **real**=총 소요시간 (user+sys+대기), **user**=사용자 영역 실행시간, **sys**=커널(시스템) 영역 실행시간
- **watch [-d] [-n 초] '명령어'**: 명령어를 **주기적으로 반복 실행**하며 결과를 갱신해서 보여줌(기본 2초 간격). **-n**: 반복 주기(초), **-d**: 이전 결과와 달라진 부분 강조. 종료는 **Ctrl+C**
- **rdate [옵션] [타임서버]**: 원격 타임 서버(NTP)에서 시간 정보를 가져와 시스템 시간과 동기화
 - **-s**: 설정만 하고 출력 안 함, **-p**: 정보만 출력하고 설정 안 함, **-u**: TCP 대신 UDP 사용
- **hwclock**: 시스템의 **하드웨어(CMOS) 시간** 정보 출력/설정 (↔ **date**는 소프트웨어/커널 시간)
- **uname [-r]**: 시스템(커널) 정보 확인. **-r**: 커널 버전
- **gzip [옵션] 파일**: 파일 압축. 압축률 **-1**(가장 빠름/낮은 압축률) ~ **-9**(최고 압축률/느림), 기본값은 **-6**. 예: `gzip -9v 파일명`
- **tar**: 여러 파일/디렉터리를 하나의 아카이브로 묶거나 푸는 유틸리티
- **du [옵션] [경로]**: 디렉터리/파일의 **사용 용량** 확인. **-s**: 요약(합계)만, **-h**: 사람이 읽기 쉬운 단위. 예: `du -sh ~`
- **df [옵션]**: 파일시스템(마운트된 디스크)의 **전체 사용량** 확인
- **quota**: 특정 사용자에게 할당된 디스크 사용량 한도 확인

9. 입출력 리다이렉션과 파이프

9.1 개념

- **리다이렉션(방향 재지정)**: 표준 입출력의 방향을 바꾸는 것. 기준은 항상 **명령어(왼쪽)**. 오른쪽(>)으로 가면 **출력**, 왼쪽으로 들어오면(<) **입력**.
- 파일 서술자(fd) 번호: **0 = 표준입력**, **1 = 표준출력**, **2 = 표준에러(오류)**

기호	의미
>	출력 전환(덮어쓰기) — 표준출력을 파일 등으로 보냄. 예: <code>cat a.txt > b.txt</code> → a.txt 내용으로 b.txt 생성(덮어씀)
>>	출력 전환(추가, append) — 기존 파일 뒤에 이어붙임. 예: <code>cat < b.txt >> a.txt</code>
<	입력 전환 — 파일 내용을 표준입력으로 사용
<<	특수 입력(Here Document) — 지정한 문자열이 나올 때까지 입력을 받아 한 번에 표준출력으로 전달
0>	표준입력을 파일로(잘 쓰이지 않는 표현)
1>	표준출력을 파일로 (>와 동일)
2>	표준에러(오류 메시지) 를 파일로 저장. 예: <code>cat nofile 2> error_log_file</code>
<code>cat < aa > bb</code> 실행 후 <code>bb</code>를 확인하면, <code>aa</code>의 내용이 그대로 <code>bb</code>로 들어가 있음(<code>aa</code> 내용과 동일하게 출력됨). <code>aa</code>가 없으면 "아무 결과도 출력되지 않음". <code>ls -al > more</code>: 현재 디렉터리 목록을 more라는 이름의 파일로 저장(<code>more</code> 명령 실행 아님! 파일명이 우연히 <code>more</code>인 것에 유의하는 함정 문제)	

9.2 파이프(Pipe, |)

- 한 명령어의 **표준출력**을 다른 명령어의 **표준입력**으로 연결하는 기술.
- 여러 명령을 조합할 때 사용. 임시 파일이 생성되지 **않음**(파이프는 메모리 버퍼를 이용 — "두 명령어 연결 시 임시 파일이 생성된다"는 지문은 **틀림**).
- 예: `ls -al | more`(목록을 화면 단위로), `who | wc -l`(현재 접속자 수), `cat /etc/passwd | grep -v linuxmaster`
- 파이프는 **여러 개를 연달아** 연결할 수 있음. 예: `ls -l | sort | less` → `ls -l`, `sort`, `less` 세 개의 프로세스가 순차적으로 입출력을 주고받으며 동작.

9.3 명령행 기호 정리

기호	의미
#	주석(root 프롬프트로도 표기됨)
~	홈 디렉터리
\$	셸 변수 참조(\$PATH 등)
&	명령을 백그라운드 로 실행
*	임의 길이의 문자열(0개 이상)을 대신하는 와일드카드
?	정확히 한 글자 를 대신하는 와일드카드 (예: <code>ls -l a?</code>)
[]	문자 범위/집합 지정(예: <code>[abc]</code>)
	파이프
<, >, >>	입력/출력(덮어쓰기)/출력(이어쓰기) 리다이렉션

기호	의미
;	명령어를 순차적으로 구분(앞 명령의 성공 여부와 무관하게 다음 명령 실행)
&&	앞 명령이 성공(종료코드 0) 해야 다음 명령 실행
	앞 명령이 실패해야 다음 명령 실행

- 예: `ls -l a* | wc -l` (a로 시작하는 모든 파일: `a`, `a.txt`, `aa`, `ab.txt`, `ac` 등 매칭 개수 세기) vs `ls -l a? | wc -l` (a+글자 1개인 파일만: `aa`, `ac`만 매칭)

10. 셸과 환경변수

10.1 내부 명령어 vs 외부 명령어

- 내부(built-in) 명령어**: 셸 자체에 내장되어 있어 셸이 직접 해석·실행. 실행 시 **별도의 새 프로세스를 만들지 않음**. 예: `cd`, `pwd`(셸 내장), `echo`, `export`, `alias`, `while`, `if`, `do` 등
- 외부(external) 명령어**: `/bin`, `/usr/bin` 등에 **파일 형태로 존재**. 실행 시 **새로운 서브 프로세스를 fork**하여 실행. (❌ "내부 명령어는 실행 시 새 서브 프로세스를 exec하여 실행한다" → 틀림, 이는 외부 명령어의 특징)
- 셸 내장 키워드가 **아닌 것** 예: `ls`(외부 명령어)

10.2 환경변수와 PATH

- `echo $PATH` (또는 `export` 만 입력) : 현재 PATH 값 확인
- PATH 추가 방법: `PATH=$PATH:[경로]` 또는 `export PATH=$PATH:[경로]`
 - 예: `export PATH=$PATH:/usr/local/bin` (기존 경로에 `/usr/local/bin`을 추가)
 - 리눅스는 경로 구분자로 **:**(콜론), 변수 앞에 **\$** 사용 (↔ 윈도우는 **;**(세미콜론), **%변수%**)
 - `export PATH=$PATH:/etc` 실행 시 → 기존 PATH에 `/etc`가 **추가됨**(기존 경로가 제거되는 것이 아님)
 - PATH는 로그인 시 자동 실행되는 **profile**류 파일에 저장해두지 않으면 로그아웃 시 초기화됨
- 셸이 명령어를 찾는 검색경로(PATH)를 지정하는 파일: `/etc/profile`(전체), 사용자 개인: `~/.bash_profile`
- TMOUT**: 일정 시간 입력이 없으면 자동 로그아웃시키는 환경변수(단위: 초, 0이면 무제한)
- alias** : 명령어 축약/재정의. 예: `alias ls='ls -aF'` (해제: `unalias ls`)
 - 현재 `ls`에 적용된 alias-경로 확인: `which ls` (alias 정의와 실제 실행 파일 경로를 함께 보여줌)
 - 매번 로그인해도 유지되게 하려면 `~/.bashrc`나 `/etc/profile`(전체 사용자)에 alias 설정을 저장해 둬 — 이 경우 `unalias`만으로는 재로그인 시 다시 살아나므로, 해당 설정 파일에서 alias 줄 자체를 지워야 완전히 제거됨
- PS1** : 프롬프트 모양 설정 / **TERM** : 터미널 종류 / **DISPLAY** : X 화면 출력 관련 (모두 `which`가 찾는 대상인 PATH와는 무관한 변수들)

10.3 기타 셸 명령어

- `source [파일]` (또는 `.` 파일) : 스크립트를 현재 셸에서 즉시 실행(번역)
- `export` : 변수를 하위 프로세스(자식 셸)에서도 사용할 수 있도록 **환경변수로 지정**
- `env` : 현재 설정된 환경변수 전체를 출력
- `set` : 변수값 설정/현재 셸의 모든 변수(환경변수+셸변수) 확인
- `declare` : 변수를 선언하며 **타입(정수·읽기전용·배열 등)** 을 함께 지정할 때 사용
- `unset [변수명]` : 변수 해제
- `history` : 사용한 명령어 목록 확인
- `whatis` : 명령어에 대한 **한 줄 요약** 설명

- **info** : GNU 프로젝트가 배포하는 온라인 매뉴얼(하이퍼텍스트 형식으로 관련 정보 제공, **man**보다 상세한 경우 多)
- 콘솔/터미널 로그아웃 방법: **exit**, **logout**, **Ctrl+D** (✗ **Ctrl+C**는 로그아웃이 아니라 현재 실행 중인 프로세스를 **중단(SIGINT)** 시키는 단축키)

11. 네트워크 명령어

명령어	설명
ifconfig [인터페이스] [옵션]	네트워크 인터페이스의 IP 주소 설정/확인. ifconfig eth0 down (비활성화), ifconfig eth1 up (활성화), -a (전체 장치 정보). MAC 주소 확인도 가능. 단, 패킷이 전달되는 경로는 ifconfig로 확인 불가 (→ traceroute)
ping [옵션] 대상	TCP/IP 기반 응용으로, 목적지 호스트에 ICMP echo request 를 보내 도달 가능 여부(응답, ICMP echo reply)를 검사. 원격 호스트의 연결 상태·대략적인 속도 확인 가능. 특정 네트워크 카드 정상 동작 여부 확인 가능. 단, "외부에서 ping을 이용해 DoS 공격을 하는지 여부"는 ping으로 확인할 수 있는 정보가 아님
traceroute 대상	목적지까지 거쳐 가는 경로(노드/홉) 를 확인. 윈도우의 tracert 에 대응. ICMP 프로토콜과 관련 있음
netstat [옵션]	네트워크 연결 상태 확인. -r (라우팅 테이블), -i (인터페이스 테이블), -s (통계), -a (모든 소켓), -p (소켓의 PID), -x (Unix domain socket 상태만)
route	라우팅 테이블 확인/추가/삭제(편집)
nslookup [-type=옵션] 도메인	네임서버에 질의. 도메인↔IP 조회. -type=mx : 메일 서버 정보 조회, -type=ns : 네임서버 정보. finger , ls , lserver 등은 nslookup 내부 명령이지만 port 는 아님
dig [@서버] [도메인] [질의타입]	Domain Information Groper. 네임서버에서 상세 DNS 정보 조회. 질의타입: a (IP), any (전체), mx (메일서버), ns (네임서버), soa
host [도메인/IP] [DNS서버]	도메인↔IP 상호 조회(nslookup보다 간단한 대체 명령)
hostname [-v 이름]	현재 시스템의 호스트명(도메인 네임) 출력/설정
wall	시스템에 로그인한 모든 사용자에게 메시지 전송
write [계정] [터미널]	특정 로그인 사용자에게 메시지 전송
talk [계정]	로그인한 사용자(원격 포함)와 실시간 대화
mesg	다른 사용자로부터 오는 메시지 수신 허용/거부 설정. mesg y (수신 허용) / mesg n (수신 거부) / 옵션 없이 실행 시 현재 상태 확인. 단, root 의 메시지는 거부해도 표시됨
mail	로그인하지 않은 사용자에게도 메시지(메일) 전달 가능

12. 시스템 관리(종료·재시작·런레벨·시간)

12.1 시스템 종료/재시작

```
shutdown [옵션] 시간 [경고메시지]
-r      : 재시작(재부팅)
-h      : 시스템 종료(정지)
-P      : 강제 종료(대문자)
-c      : 예약된 shutdown 명령 취소
-k      : 실제 종료는 하지 않고 메시지만 출력
-f      : shutdown 전 수행 중인 모든 프로세스에 kill 시그널 전송
+m      : m분 후 실행
```

- `shutdown -h +10` : 10분 후 시스템 종료
- `shutdown -r +10` : 10분 후 재부팅
- `shutdown -h 20:00` : 오후 8시에 종료
- `shutdown -h 00:00` : 자정(밤 12시)에 종료
- 재부팅과 결과가 같은 것: `reboot`, `shutdown -r`, `init 6(=telinit 6)`
- 종료와 결과가 같은 것: `shutdown -h now`, `halt`, `init 0`, `poweroff`
- 로그아웃(연결 종료, 시스템은 켜진 채)과 시스템 종료(전원 off)는 다른 개념임에 유의.

12.2 프로세스/실행 관련 짧은 명령

- `ps -ef | grep [프로세스명]` : 특정 프로세스 정보만 확인. 예: `ps -ef | grep xinetd`
- `top` : CPU 사용률·메모리·프로세스 상태를 **지속적으로 모니터링**할 때 사용하는 대표 도구
- `pstree` : 프로세스 간 상관관계(부모-자식)를 트리 형태로 확인
- `free` : 시스템의 메모리 사용량 확인
- `nice` / `renice` : 프로세스 우선순위 지정/변경
- `kill [-시그널번호] PID` : 프로세스에 시그널 전송
 - **SIGTERM(15)** : 프로세스에게 스스로 안전하게 종료하라고 요청하는 시그널(기본값, 강제 종료 아님)
 - **SIGINT(2)** : `Ctrl+C`로 보내는 인터럽트(실행 중지) 시그널
 - **SIGKILL(9)** : 프로세스를 **즉시 강제 종료**(정리 작업 없이 강제 종료되므로 최후의 수단)
- `sleep [초]` : 지정한 시간만큼 실행을 지연. 예: `cd /home; sleep 10; ls /home`
- `uname [옵션]` : 시스템(커널) 정보 확인
 - `-a` : 모든 정보, `-s` : 커널 이름만, `-r` : 커널 릴리즈(버전) 번호만, `-v` : 커널 빌드 날짜 등 종합 정보

12.3 패키지 관리 도구

배포판 계열마다 사용하는 패키지 관리 도구가 다릅니다.

배포판 계열	저수준(직접 설치/삭제) 도구	고수준(의존성 자동 해결) 도구
레드햇 (RHEL/CentOS/Fedora)	RPM (RPM Package Manager)	YUM (최근 배포판은 DNF)
데비안(Debian/Ubuntu)	dpkg (보조: dselect, alien)	apt-get → aptitude

배포판 계열	저수준(직접 설치/삭제) 도구	고수준(의존성 자동 해결) 도구
수세(SUSE)	YaST(Yet Another Setup Tool, 설치·구성 통합 도구)	Zypper(Zypp)

12.4 세션/셸 종료 방법 정리

- 터미널/콘솔 로그아웃: `exit`, `logout`, `Ctrl+D`
- 강제 종료(중지) 단축키: `Ctrl+C` (SIGINT — 로그아웃과는 다른 개념)

13. 자주 나오는 헛갈리는 포인트 정리

시험에 반복적으로 등장하는 "함정" 지문들을 모아 정리합니다.

1. 배포판 설치 유형에 "메인프레임"은 없다. (워크스테이션/서버/사용자정의/업그레이드만 있음)
2. 리눅스 커널은 C 언어로 작성되었다. ("COBOL로 작성" → 틀림)
3. GNU 프로젝트는 UNIX를 개발한 프로젝트가 아니다. (UNIX는 벨 연구소가 개발)
4. `/etc/passwd`의 패스워드 필드가 x인 것은, 실제 패스워드가 `/etc/shadow`에 암호화되어 저장되었기 때문이다. (`/etc/passwd`에 암호화되어 저장된 게 아님)
5. 주 파티션은 최대 4개, 확장 파티션은 1개만, 논리 파티션은 확장 파티션 안에 최대 12개까지, 논리 파티션 번호는 5번부터.
6. `/opt`는 응용프로그램 설치 공간이지, socket·log 파일 저장 공간이 아니다(그건 `/var`).
7. `/include`는 리눅스 표준 디렉터리가 아니다.
8. 파이프(|)는 임시 파일을 만들지 않는다.
9. `touch`는 파일이 있으면 내용은 그대로 두고 시간 정보만 갱신하며, 파일이 없으면 빈 파일을 새로 만든다. Change Time은 임의로 과거로 바꿀 수 없다.
10. `mv`는 디렉터리 대상 작업 시 별도 옵션이 필요 없다(`cp/rm`은 `-r` 등이 필요).
11. 저널링 파일시스템: ext3, ext4, JFS, XFS, ReiserFS (**ext2는 저널링 아님**).
12. 네트워크 파일시스템: SMB, NFS, CIFS (**JFS는 로컬 파일시스템**이지 네트워크 파일시스템이 아님).
13. **NFS는 파일 공유용이지 사용자 "인증" 시스템이 아니다.** (인증 시스템: NIS, LDAP, Kerberos)
14. 소스 코드 수정 시에도 공개 의무가 없는 라이선스: BSD, Apache, MIT (반대로 GPL 계열은 공개 의무 있음, LGPL·MPL은 부분 공개 의무).
15. **CentOS는 RHEL의 소스를 그대로 가져와 만든 무료 복제본이다.**
16. **which**는 실행파일의 위치만, **whereis**는 실행파일+소스+man 페이지 위치까지 찾아준다.
17. **more**는 아래로만 스크롤, **less**는 위아래 모두 자유롭게 스크롤 가능.
18. **GRUB [a]**=커널 매개변수 추가, **[c]**=명령줄(대화식) 모드, **[e]**=부트 메뉴 항목 편집.
19. 런레벨 0=종료, 1=단일 사용자(복구용), 3=다중 사용자 텍스트, 5=다중 사용자 그래픽, 6=재부팅.
20. **cd -**는 직전 디렉터리로, **cd ..**는 부모 디렉터리로, **cd**(또는 **cd ~**)는 홈 디렉터리로 이동한다.
21. 스왑 파티션 ID는 82.
22. **E-IDE**를 **Secondary Slave**로 연결하면 **/dev/hdd**로 인식된다(파티션 번호 없이).
23. **passwd** 실행 시 새 패스워드는 화면에 표시되지 않는다.
24. **cal**을 옵션 없이 실행하면 이번 달 달력만 출력된다(1년 전체 아님).
25. 리눅스는 최초 버전(0.01)부터 **SMP**(멀티프로세서)를 지원한 것이 아니라, **1996년 커널 2.0**부터 지원했다. 1991년 최초 공개 시점에는 네트워킹 기능도 없었다.
26. **KDE**는 **GNU 프로젝트** 소속이 아니다(GNOME은 GNU 프로젝트 소속).

27. 소스 코드 수정 시에도 공개 의무가 없는 라이선스: BSD, Apache, MIT. **MPL**은 수정한 그 코드만 공개 의무가 있고, 결합된 다른 코드/실행파일은 공개하지 않아도 된다(부분 공개).
28. **CentOS 7**부터 기본 파일시스템은 **xfs**이다(ext4가 아님에 주의).
29. **LVM** 구성 순서는 **PV** → **VG** → **LV**이다.
30. **/etc/shadow**에는 **UID·GID·로그인 셸 정보**가 없다(이는 **/etc/passwd**의 필드).
31. **cp -f**는 확인 없이 덮어쓰기, **cp -i**는 덮어쓰기 전에 확인을 묻는다 — 서로 반대 개념이므로 혼동 주의.
32. 파이프(**|**)는 여러 개를 이어서 연결할 수 있으며(**A | B | C**), 임시 파일을 만들지 않는다.
33. **cd, cd ~, cd \$HOME**은 모두 홈 디렉터리로 이동하지만, **cd HOME**은 그런 이름의 디렉터리가 없다는 오류가 난다(대소문자·\$ 유무로 의미가 완전히 달라짐).
34. **PATH** 등 환경변수 구분자는 리눅스가 **:**(콜론), 윈도우는 **;**(세미콜론) / 변수 참조는 리눅스가 **\$변수명**, 윈도우는 **%변수명%**.
35. **diff -i**는 대소문자를 "구별하지 않고"(무시하고) 비교하는 옵션이다 — "대소문자를 구별한다"는 설명은 정 반대라 틀린 지문.
36. **Knoppix**는 **데비안(Debian)** 기반이다 — 슬랙웨어 계열 목록에 낀 보기로 나오면 그게 오답(정답 아님).
37. **홈 디렉터를 하위 디렉터리까지 포함해 삭제하는 **userdel** 옵션은 소문자 **-r****이다(대문자 **-R**이 아님에 주의 — 대문자 **-R**은 실제로는 chroot 경로 지정 옵션).

참고: 이 문서의 근거 자료

[resources/](#) 폴더의 다음 자료를 모두 확인하여 종합 작성했습니다.

파일	특징
linux_master_2.html	네이버 블로그 "리눅스마스터 2급 1차 시험 족보" (2026-01-27 수정본). 문항별 상세 해설(▶해설) 포함, 가장 최신이고 근거가 가장 풍부한 핵심 자료
(pdf)리눅스2급1차족보new.pdf, (pdf)리눅스2급1차족보 new (1).pdf, 리눅스 2급 1차 족보.pdf, 리눅스 마스터 2급 1차 족보.pdf	동일 계열의 과거 기출 압축 정리 PDF(서로 대부분 중복). 문항 뒷부분에 1~67번 이론 요약 섹션 (디렉토리 구조, 부팅, 파티션, 명령어 사전 등) 포함
리눅스마스터2급1차.docx, 리눅스마스터2급1차.txt	위 PDF와 동일한 기출 50문항 세트(중복 확인됨)
리마2급총정리.hwp, 리눅스_마스터_2급_1차_완전총정리-vstu77.hwp, 리눅스마스터2급 1차 족보1.hwp, 리눅스마스터2급 1차 족보2.hwp	HWP 내부 미리보기 텍스트(PrvText 스트림) 확인 결과, 위 PDF의 이론 요약·기출 문항과 동일한 내용 으로 확인되어 별도 반영(중복 자료)

- HTML 자료는 총 3,126줄 분량으로, 배경 조사 후 전체(1~3126행)를 이론·수치·옵션 단위로 정리하여 반영했습니다.
- 원본 자료들은 여러 해에 걸쳐 작성된 기출 모음이라, 일부 지문(LILO의 BIOS 참조 여부, GRUB의 부트 디스크 지원 여부, GRUB [c] 키의 정확한 의미 등)에서 **자료 간 서술이 서로 다른 경우**가 있었습니다. 이 문서는 **가장 최신 (2026년 개정) 해설을 우선**하여 정리하되, 자료 간 차이가 있는 부분은 본문에 함께 표시해 두었습니다.